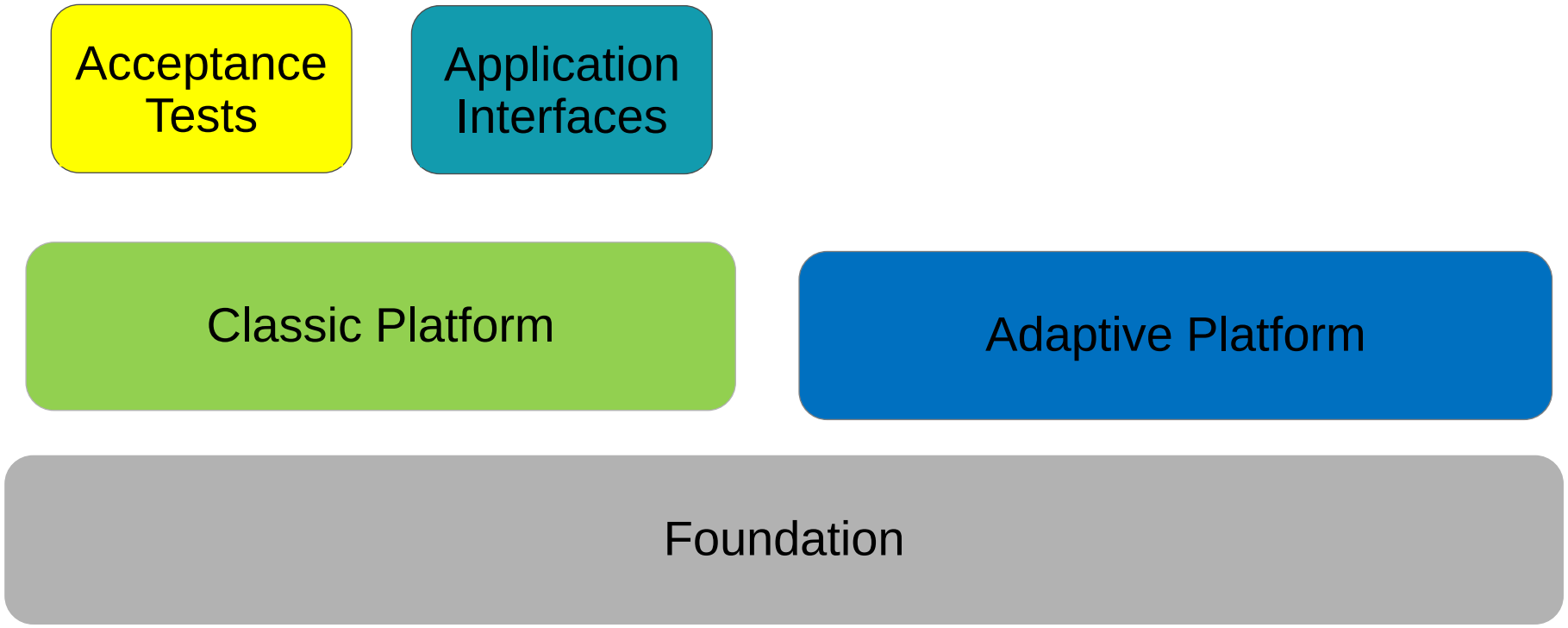
AUTOSAR Products

AUTOSAR Products



AUTOSAR Products

Application Interfaces	▪ API for vehicle domains: Body & Comfort, Transmission, Powertrain, Chassis, Occupant & Pedestrian Safety, HMI, Multimedia, and Telematics ▪ Current Version: R20-11 (Release: November 2020)
Acceptance Tests	▪ System test at bus level and application level to validate the behavior of the stack ▪ Current Version: R1.2 (Release: December 2016)
Classic Platform	▪ Non-flexible architecture and static binding ▪ Used for real-time and safety critical systems, i.e., Engine and Airbag ▪ Current Version: R20-11 (Release: November 2020)
Adaptive Platform	▪ Designed to complement Classic Platform for applications with high demand of computation power but less real-time and safety criticality ▪ Runtime binding (implements AUTOSAR Runtime for Adaptive Applications (ARA)) ▪ Current Version: R20-11 (Release: November 2020)
Foundation	▪ Relevant for both Classic and Adaptive Platform ▪ Network Management, Debugging, E2E, etc. ▪ Current Version: R20-11 (Release: November 2020)

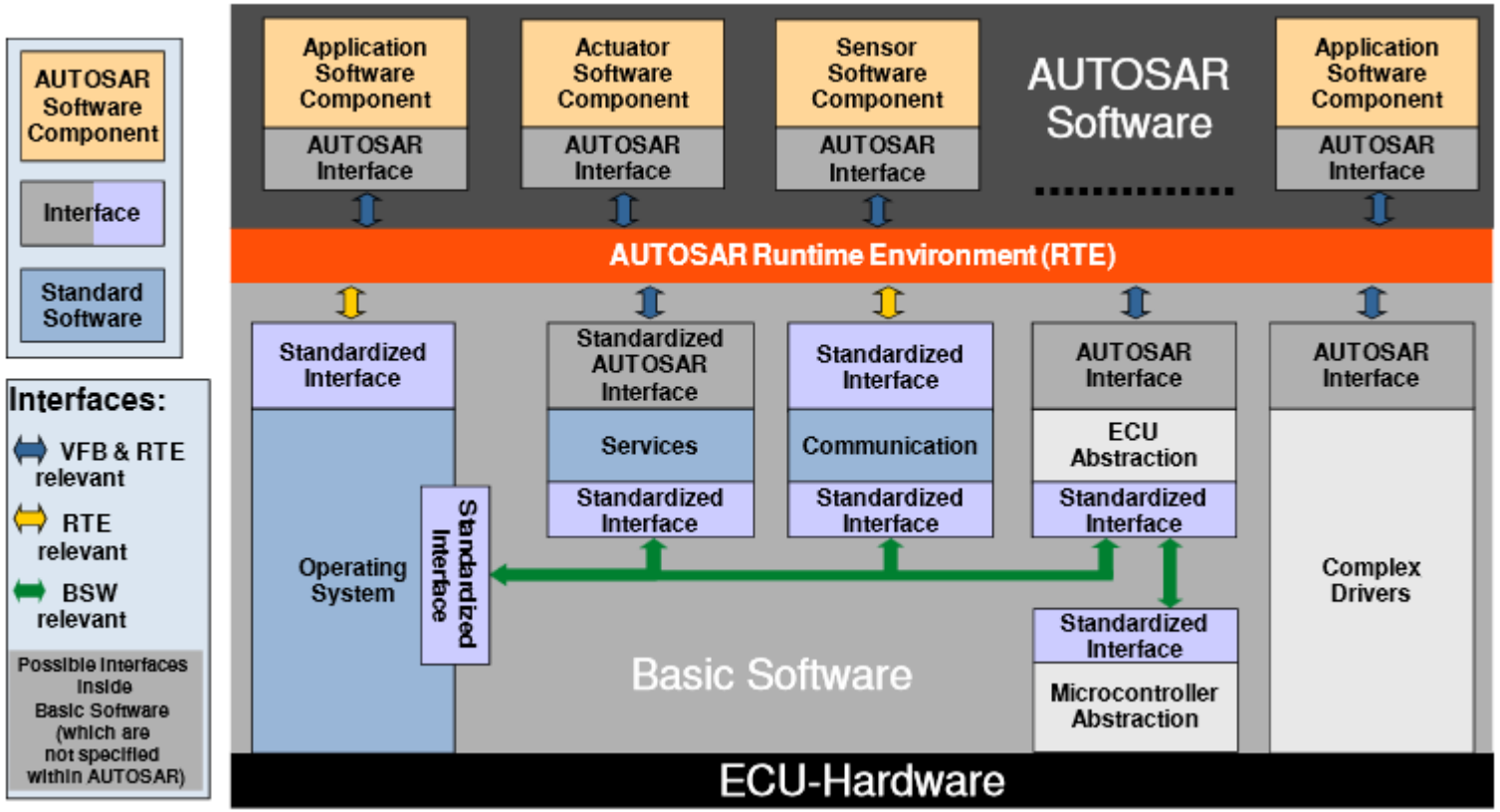
Classic vs. Adaptive Platform:
Technical Characteristics

AUTOSAR Classic Platform	AUTOSAR Adaptive Platform
Based on OSEK	Based on POSIX
Execution of code directly from ROM	App. is loaded from persistent memory into RAM
Same address space for all applications (MPU support for safety)	Each application has its own (virtual) address space (MMU support)
Optimized for signal-based communication (CAN, FlexRay)	Service-oriented communication
Fixed task configuration	Support of multiple (dynamic) scheduling strategies
Specification	Specification and code

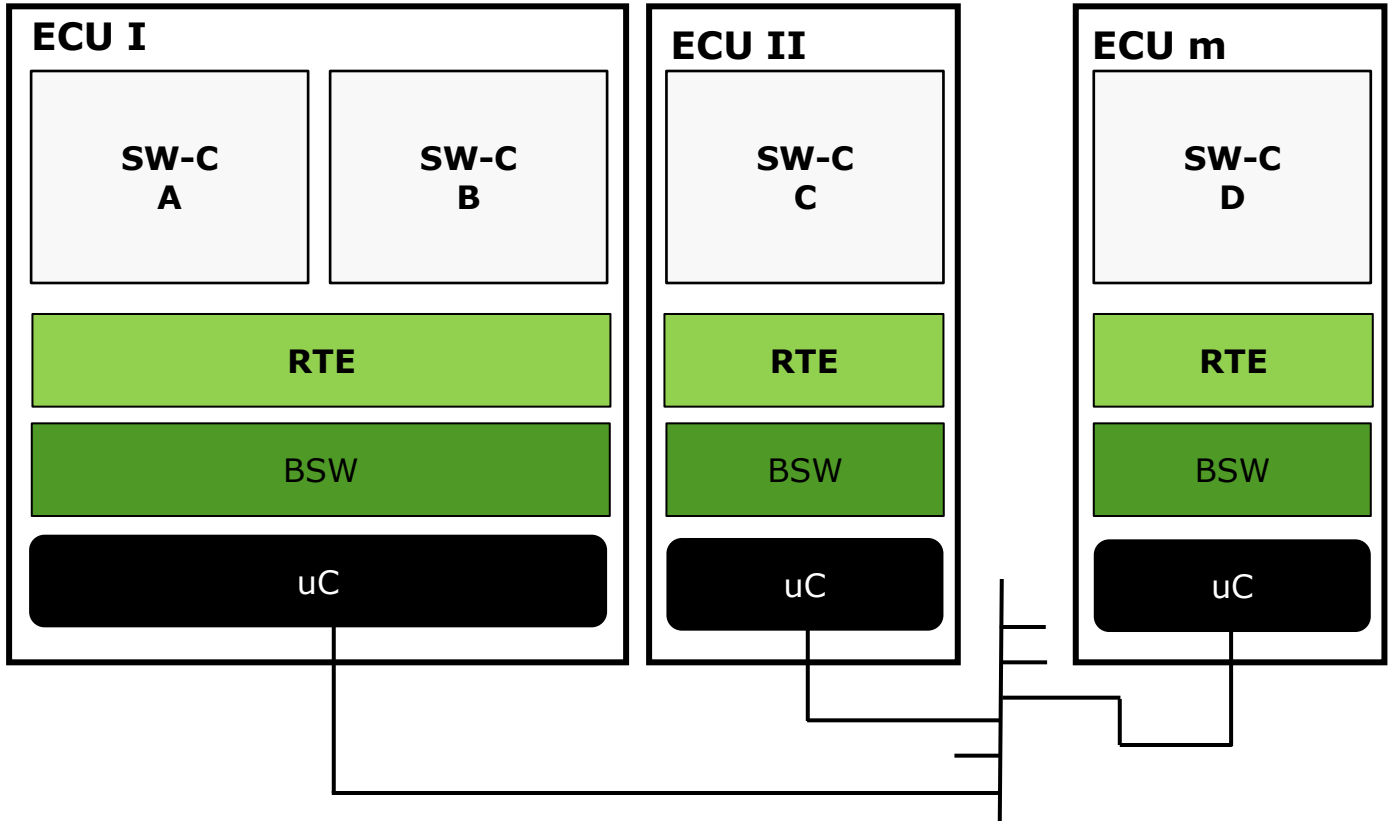
AUTOSAR Concepts

AUTOSAR Concepts – Basic AUTOSAR Glossary

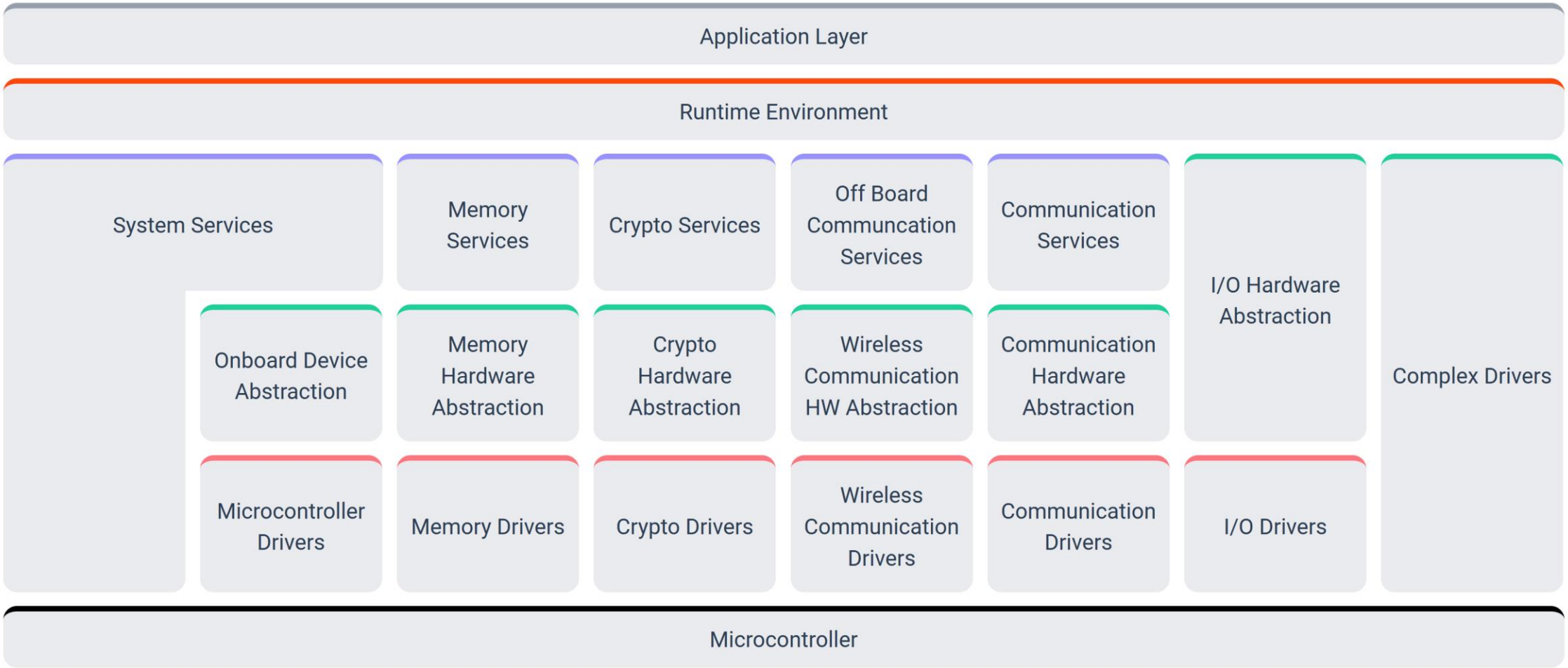
- ***Software components (SW-C)***
  – A piece of software to be run in an AUTOSAR system
- ***Virtual Functional Bus (VFB)***
  – High level communication abstraction
- ***Run Time Environment (RTE)***
  – Implements the VFB on one ECU
- ***Basic Software (BSW)***
  – Standard software for standard ECU functionality (OS, communication, memory, hardware drivers, diagnostics etc.)



The AUTOSAR Architecture

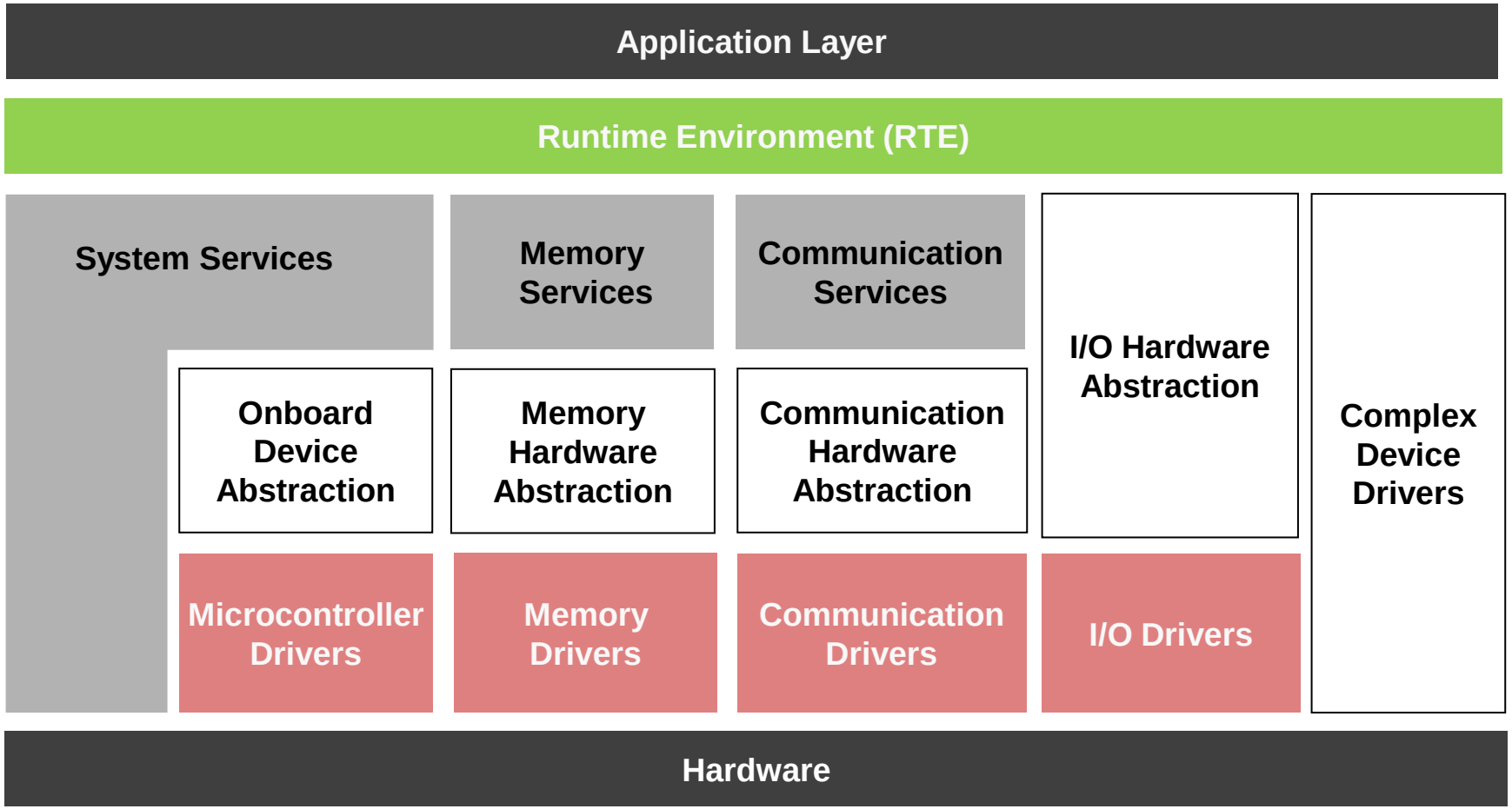


The AUTOSAR Classic Platform (Release: R20-11)

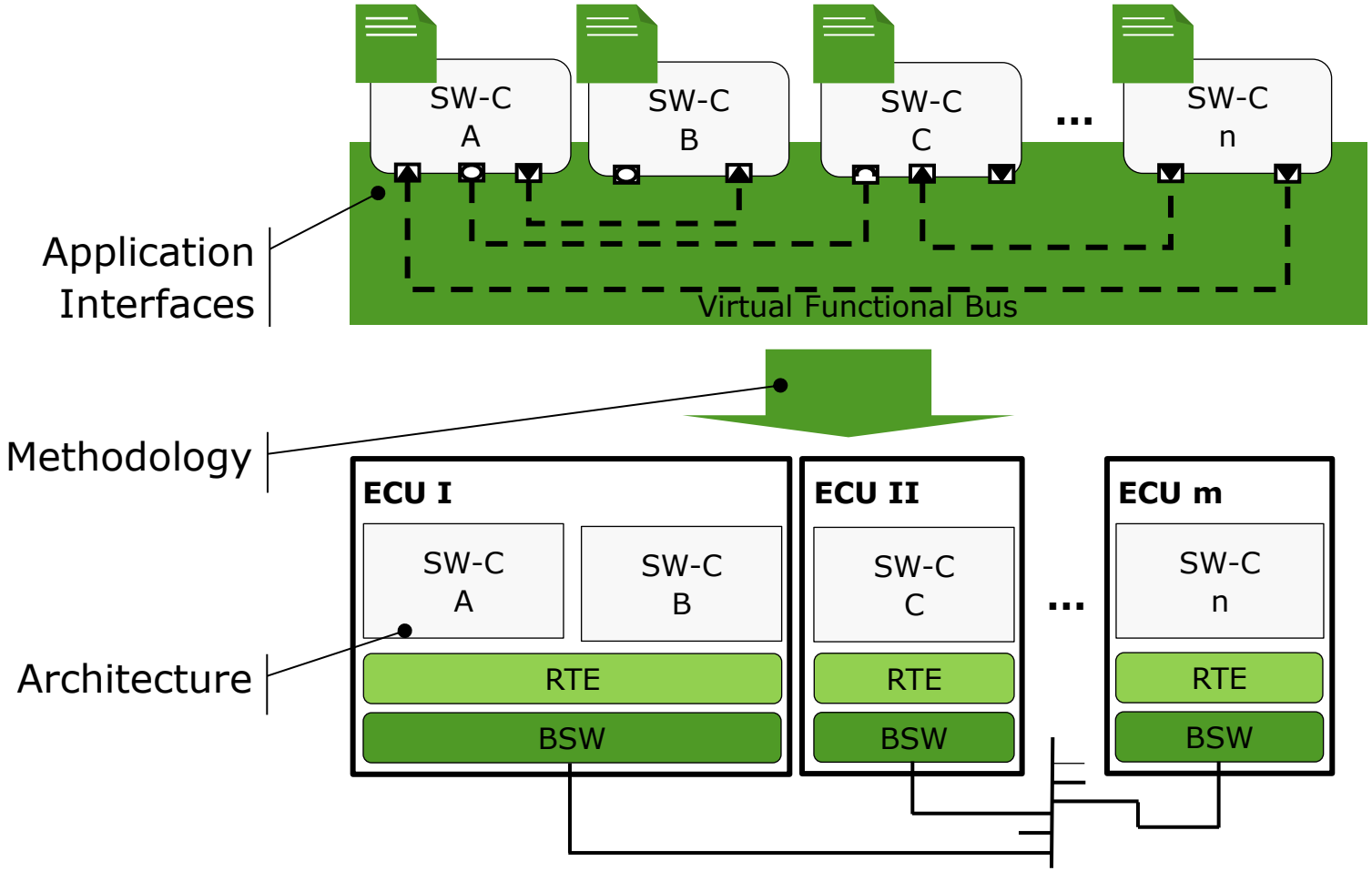


Source: https://www.autosar.org/standards/classic-platform/

The AUTOSAR Architecture

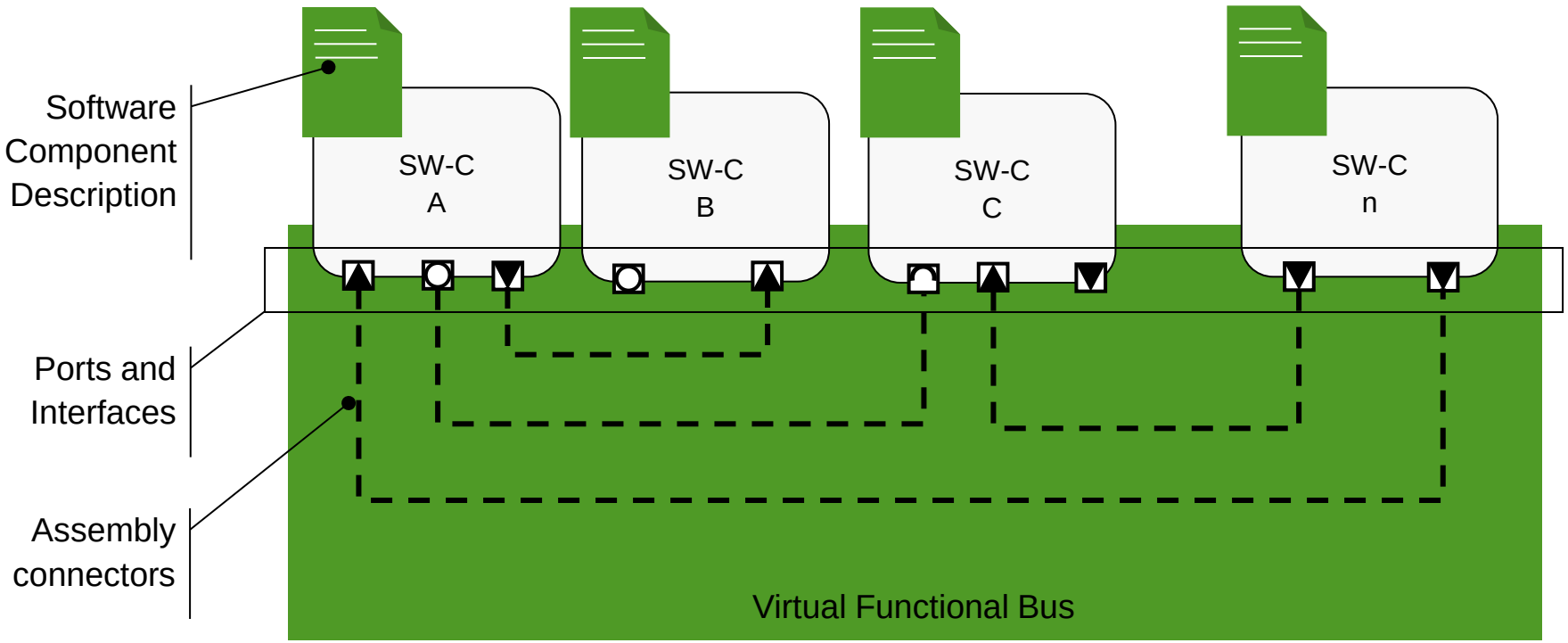


The AUTOSAR Architecture – Software Components and Virtual Functional Bus



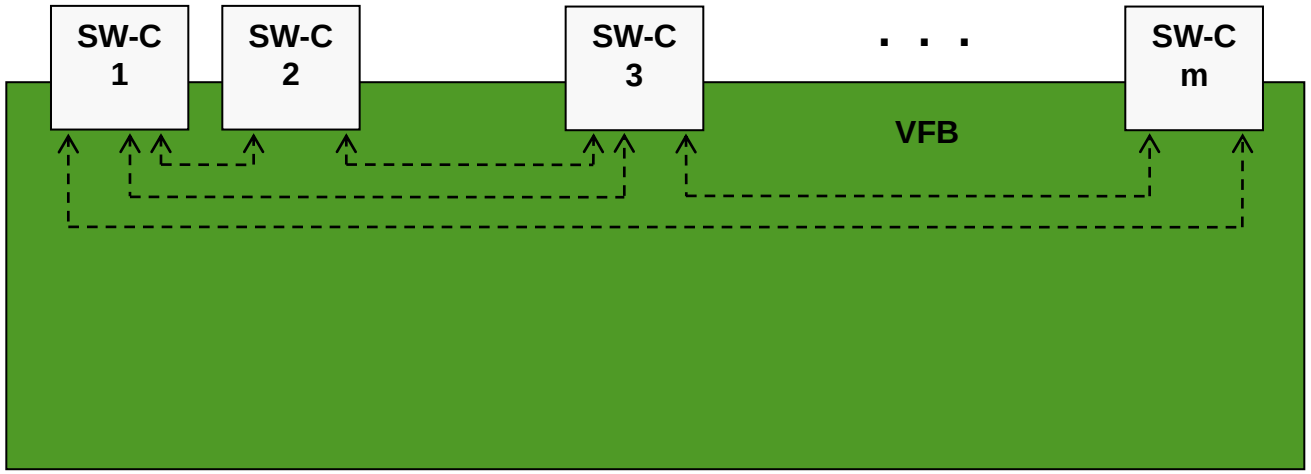
The AUTOSAR Architecture – Software Components and Virtual Functional Bus

- SW-C communicates through well defined ***Ports*** and ***Interfaces***
- Ports and Interfaces are specified in ***Software Component Descriptions***



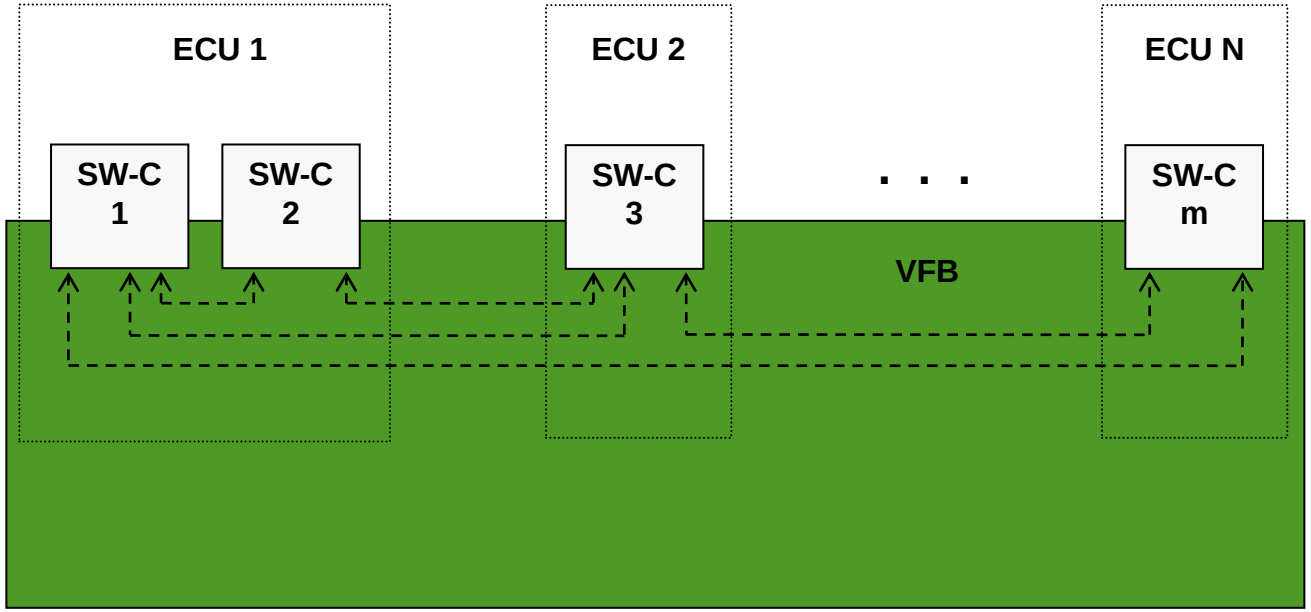
The AUTOSAR Architecture – Virtual Functional Bus and Runtime Environment

- A ***Software Component*** (SW-C) is an application module that implements an AUTOSAR application
- An SW-C is a high level abstraction which is unaware of where in the system (in which ECU) it is situated
- The ***Virtual Functional Bus*** (VFB) is a high level abstraction of the available communication paths between the SW-Cs



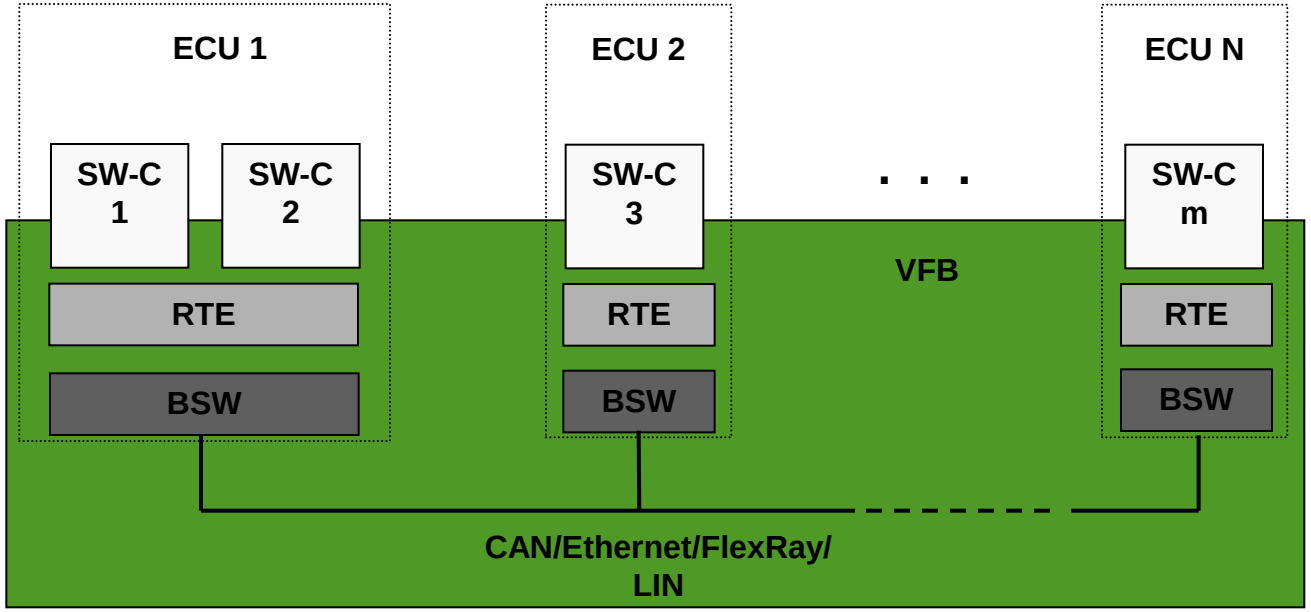
The AUTOSAR Architecture – Virtual Functional Bus and Runtime Environment

- During system design, the SW-Cs are partitioned onto ECUs
- There are two different types of communication paths in the VFB
  - ***Intra-ECU*** (inside one ECU)
  - ***Inter-ECU*** (between different ECUs)



The AUTOSAR Architecture – Virtual Functional Bus and Runtime Environment

- The ***Runtime Environment*** (RTE) is the only interface to the SW-Cs
- The RTE implements the VFB
- The RTE uses CAN/Ethernet/FlexRay/LIN buses for inter-ECU communication via the ***Basic Software Layer*** (BSW)

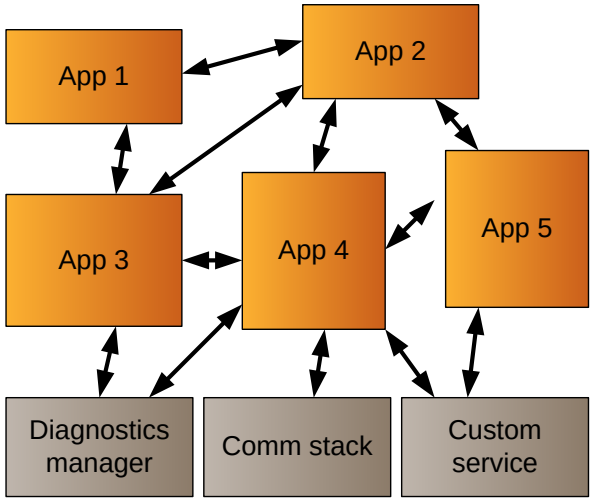


The AUTOSAR Architecture – Virtual Functional Bus and Runtime Environment

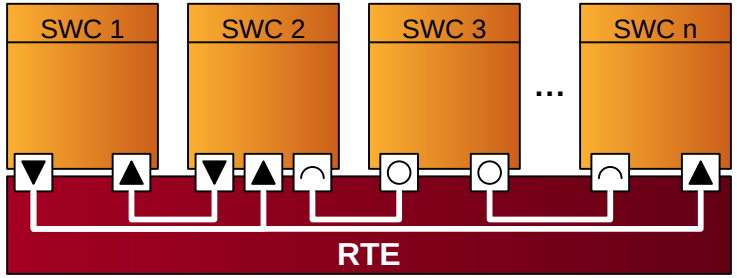
- AUTOSAR is a standardized platform
- Well defined APIs for communication
- No difference between internal communication and bus communication

→ ***Relocatability and reuse!***

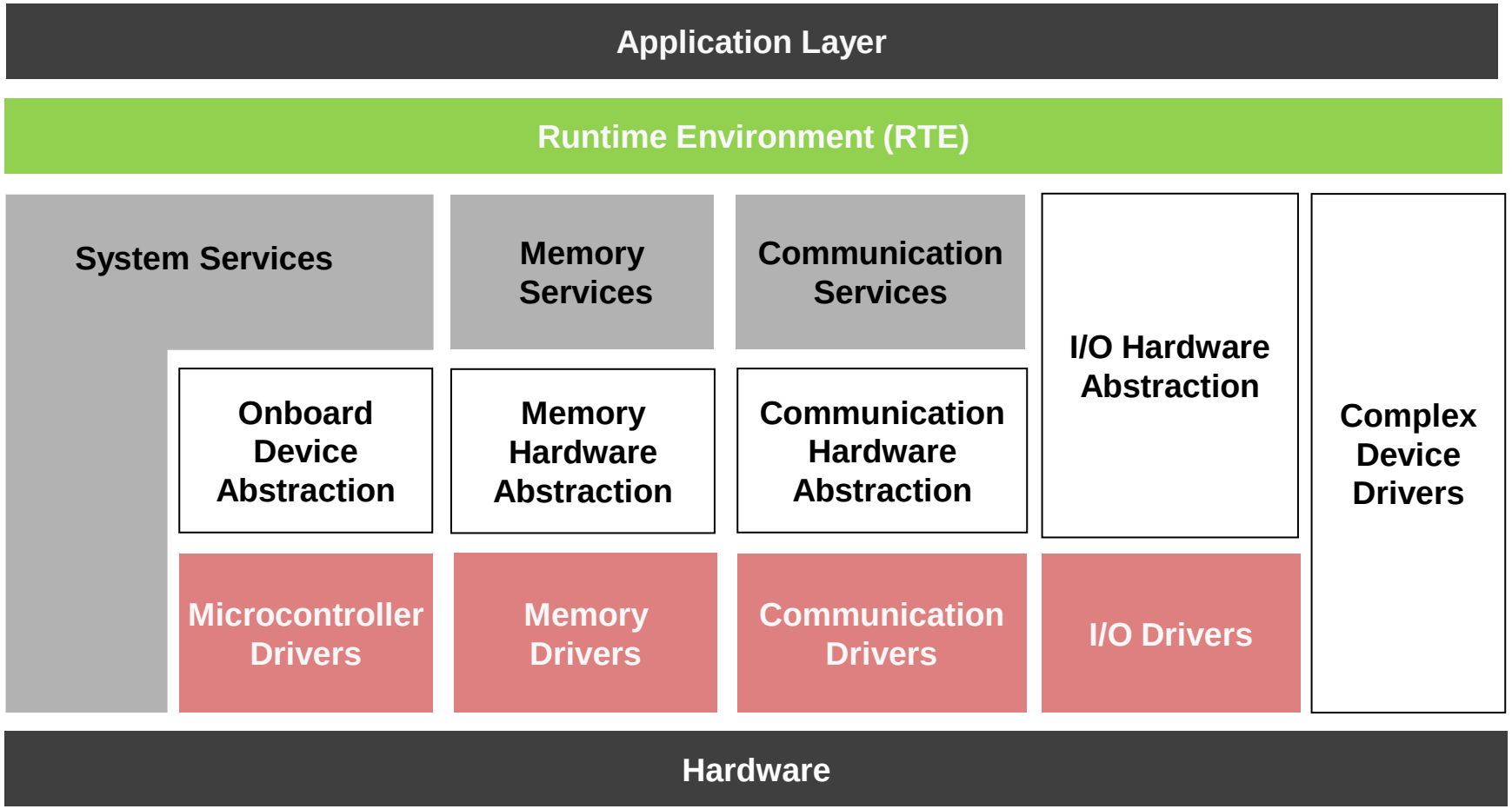
Yesterday



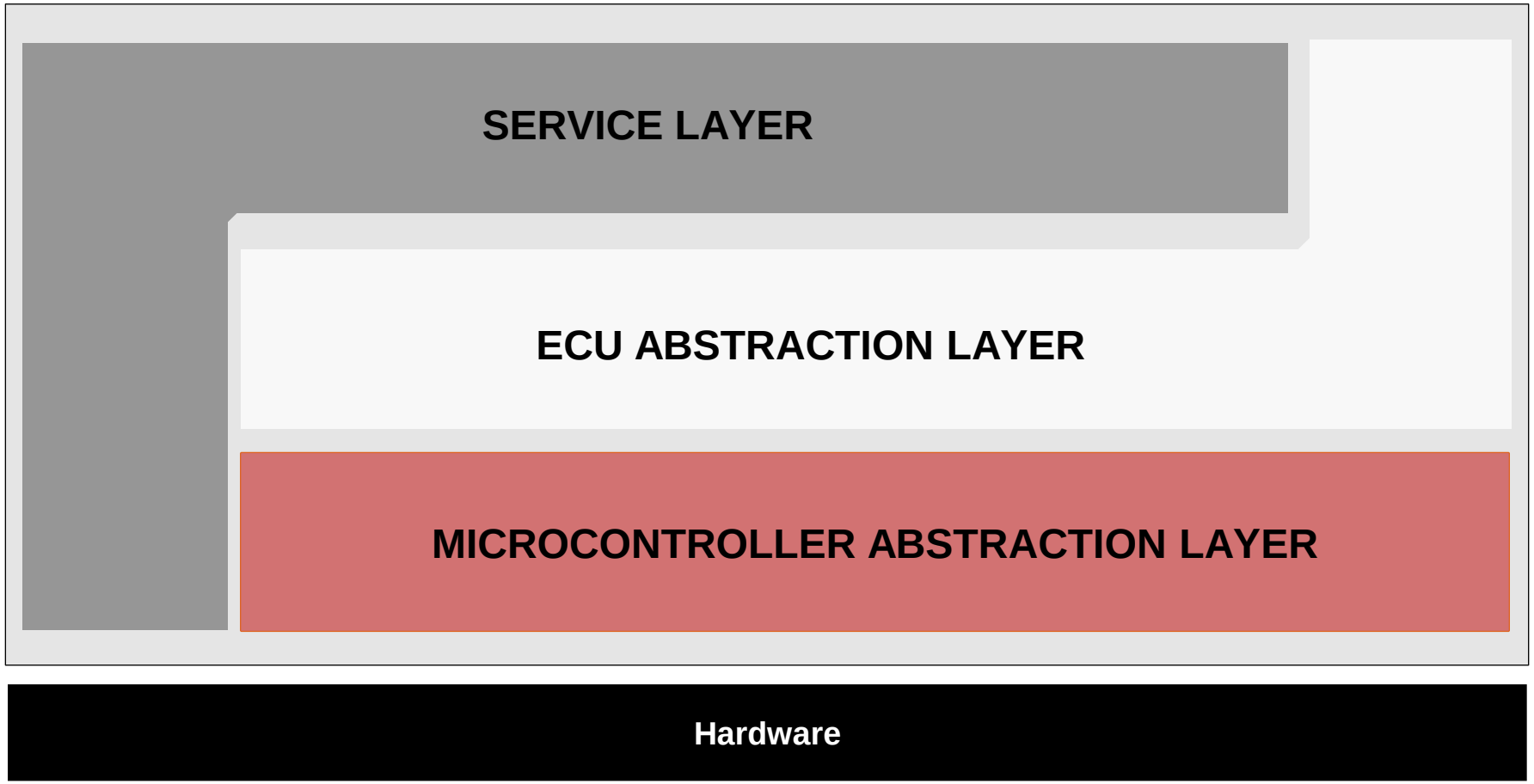
AUTOSAR



The AUTOSAR Architecture

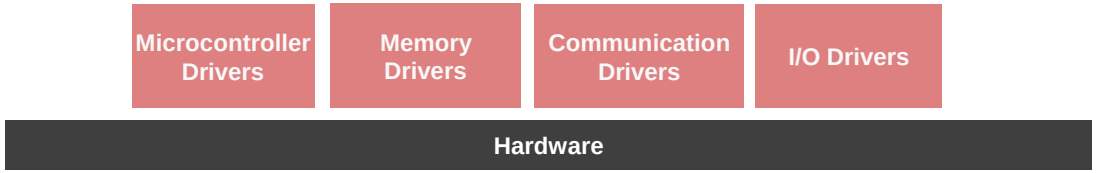


The AUTOSAR Architecture – Basic Software



The AUTOSAR Architecture – MCAL

- The ***Microcontroller Abstraction Layer*** is the lowest software layer of the Basic Software. It contains internal drivers, which are software modules with direct access to the µC internal peripherals and memory mapped µC external devices.

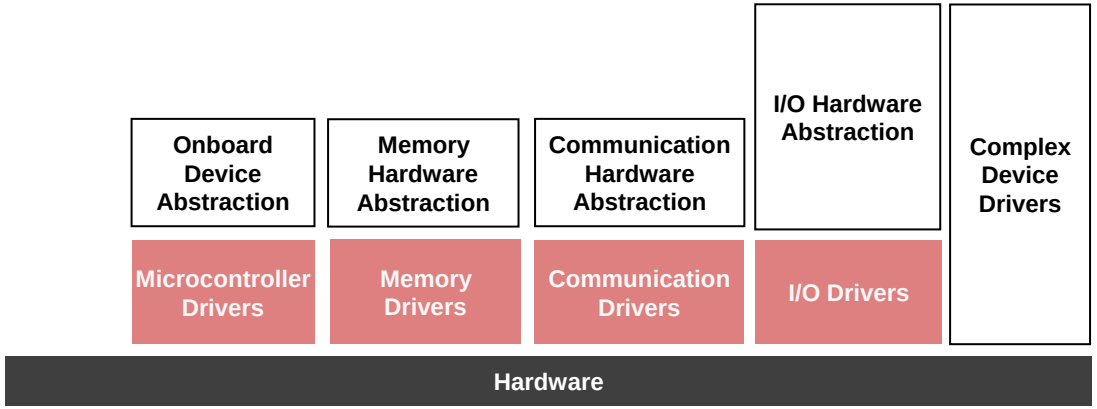


- Task:
  - Make higher software layers independent of µC

- Properties:
  - Implementation: µC dependent
  - Upper Interface: Standardized and µC independent

The AUTOSAR Architecture – ECU Abstraction Layer

- The ***ECU Abstraction Layer*** interfaces the drivers of the Microcontroller Abstraction Layer. It also contains drivers for external devices. It offers an API for access to peripherals and devices regardless of their location (µC internal/external) and their connection to the µC (port pins, type of interface)

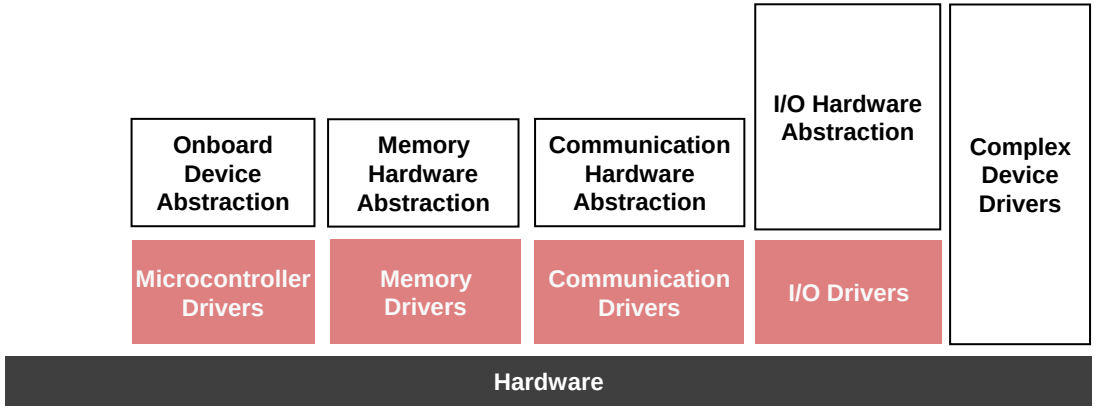


- Task:
  - Make higher software layers independent of ECU hardware layout, e.g. bus types, memory devices

- Properties:
  - Implementation: µC independent, ECU hardware dependent
  - Upper Interface: µC and ECU hardware independent, dependent on signal type

The AUTOSAR Architecture – Complex Device Driver

- A ***Complex Device Driver*** implements complex sensor evaluation and actuator control with direct access to the µC using specific interrupts and/or complex µC peripherals (like PCP, TPU), e.g.
  - Injection control
  - Electric valve control
  - Incremental position detection

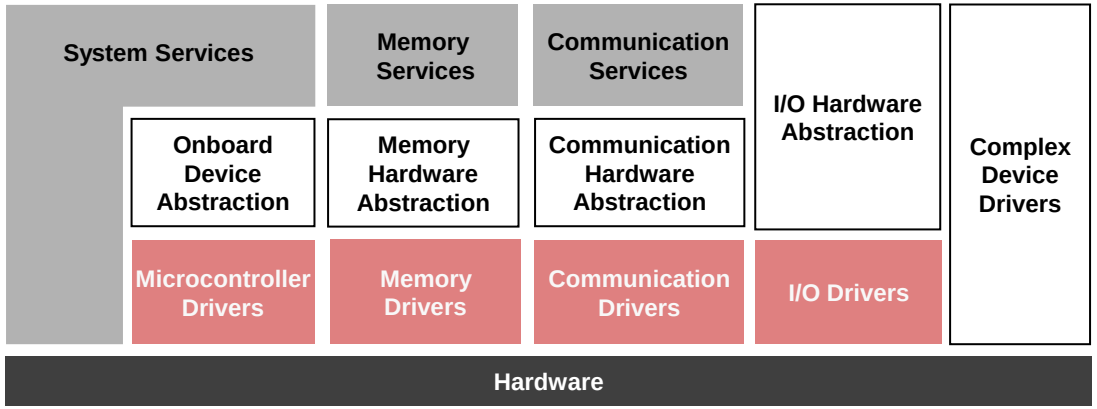


- Task:
  - Fulfill the special functional and timing requirements for handling complex sensors and actuators

- Properties:
  - Implementation: Highly µC, ECU and application dependent
  - Upper Interface: Specified and implemented according to AUTOSAR (AUTOSAR interface)

The AUTOSAR Architecture – Service Layer

- The ***Service Layer*** is the highest layer of the Basic Software which also applies for its relevance for the application software: while access to I/O signals is covered by the ECU Abstraction Layer, the Services Layer offers:
  - Operating system functionality
  - Vehicle network communication /management
  - Memory services (NVRAM management)
  - Diagnostic Services (UDS, OBD)
  - Mode management

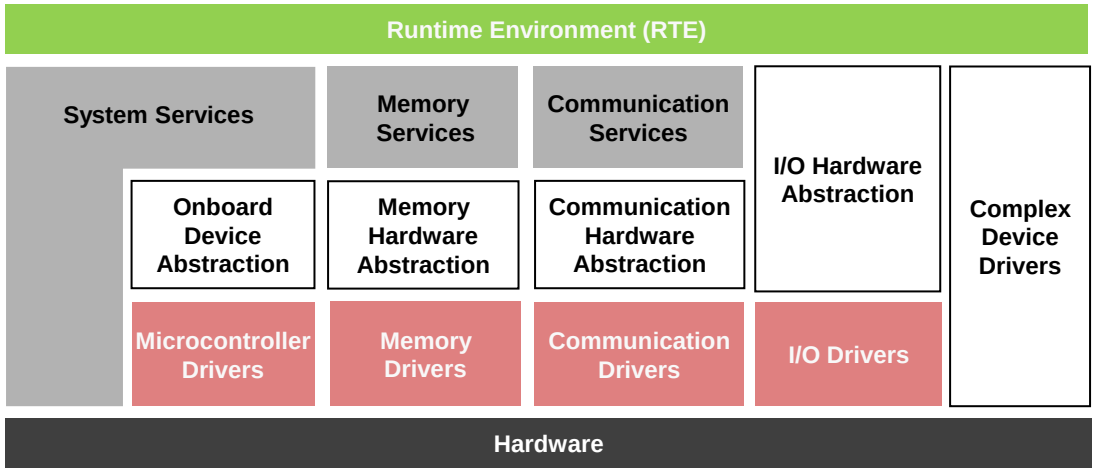


- Task:
  - Provide basic services for application and basic software modules.

- Properties:
  - Implementation: Partly µC, ECU hardware and application specific
  - Upper Interface: µC and ECU hardware independent

The AUTOSAR Architecture – Runtime Environment (RTE)

- The ***RTE*** is a layer providing communication services to the application software (AUTOSAR Software Components and/or AUTOSAR Sensor/Actuator components).
  Above the RTE the software architecture style changes from “layered“ to “component style“. The AUTOSAR Software Components communicate with other components (inter and/or intra ECU) and/or services via the RTE.



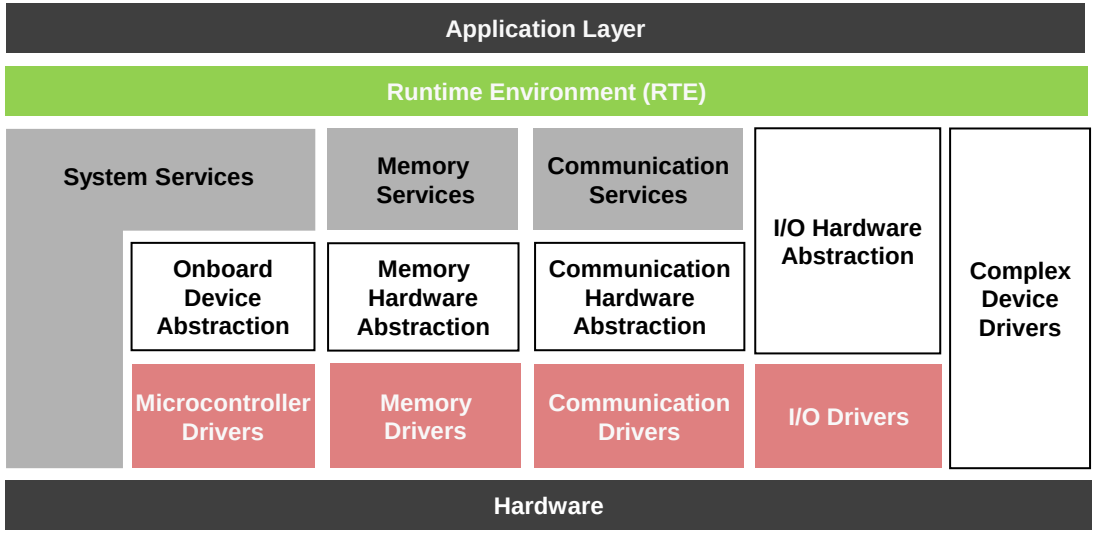
- Task:
  - Make AUTOSAR Software Components independent from the mapping to a specific ECU

- Properties:
  - Implementation: ECU and application specific (generated individually for each ECU)
  - Upper Interface: Completely ECU independent

The AUTOSAR Architecture – Application Layer

- The ***Application Layer*** is a layer providing application software (AUTOSAR Software Components and/or AUTOSAR Sensor/Actuator components).

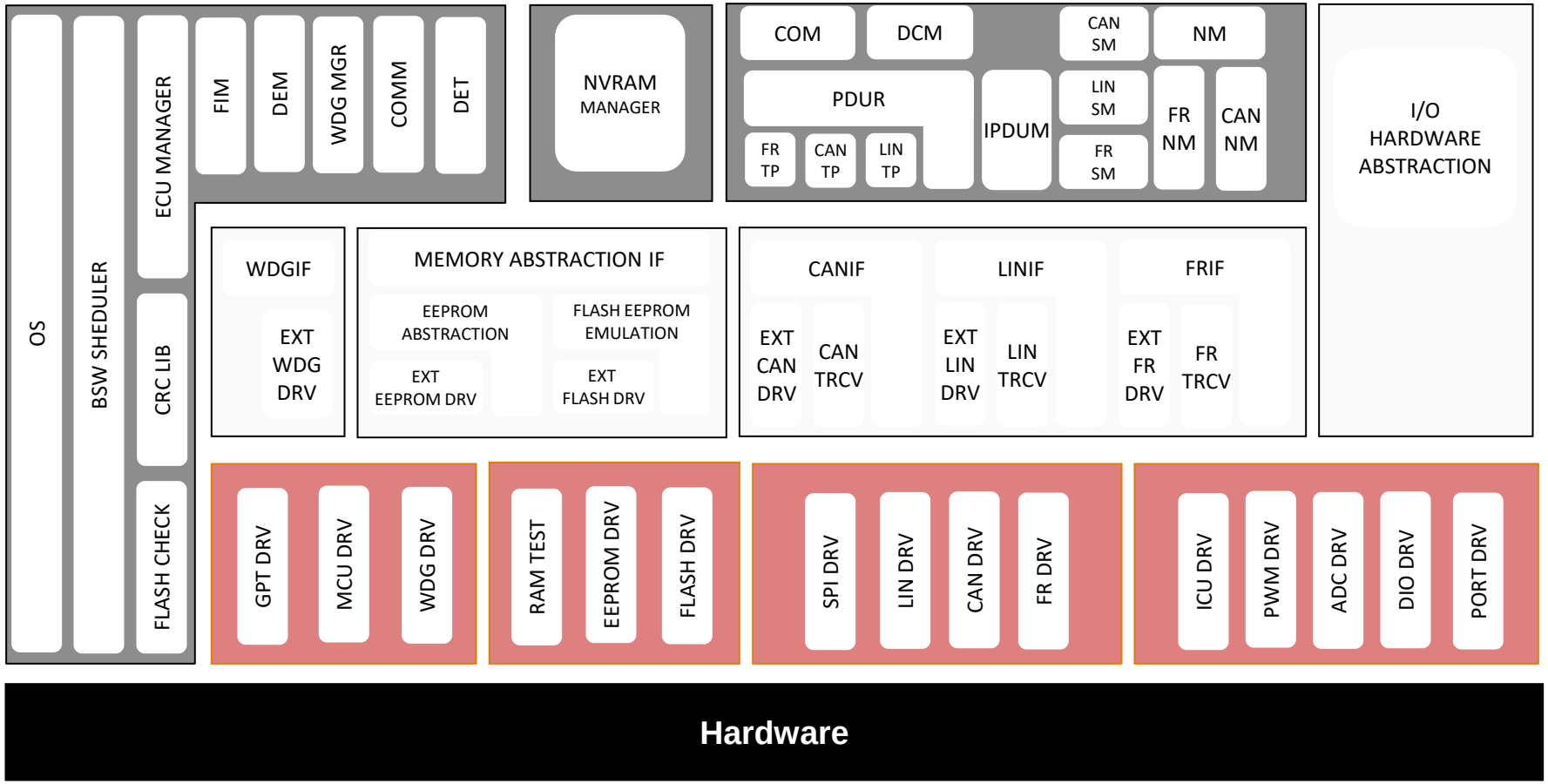
  Above the RTE the software architecture style changes from “layered“ to “component style“. The AUTOSAR Software Components communicate with other components (inter and/or intra ECU) and/or services via the RTE.



- Task:
  - Implement applications (runnables) that are executed by the RTE

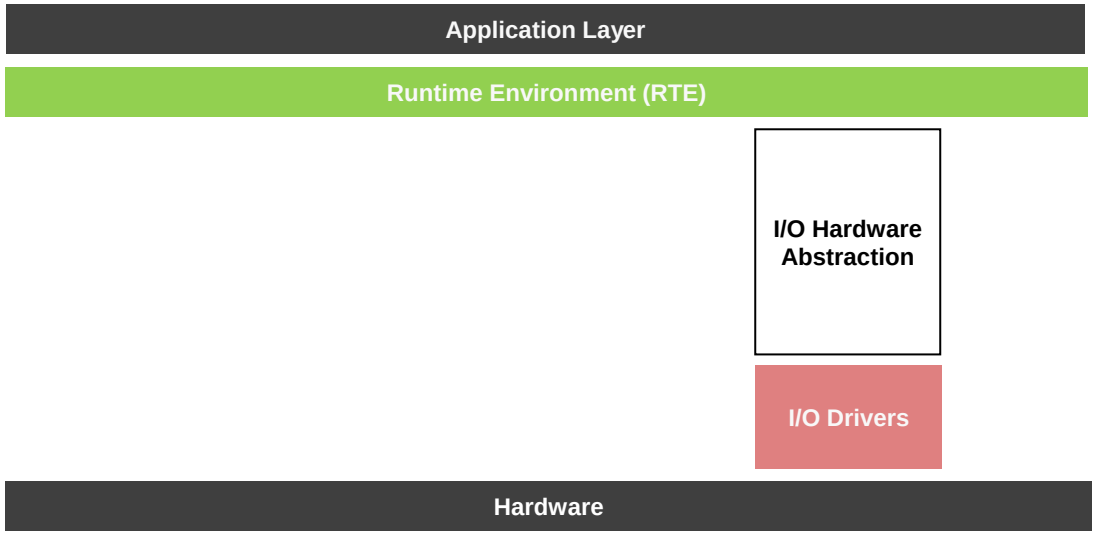
- Properties:
  - Applications completely ECU independent.
  - Sensor/Actuator SW-Cs are dependent on the specifics of a sensor or actuator

The AUTOSAR Architecture – Basic Software Modules



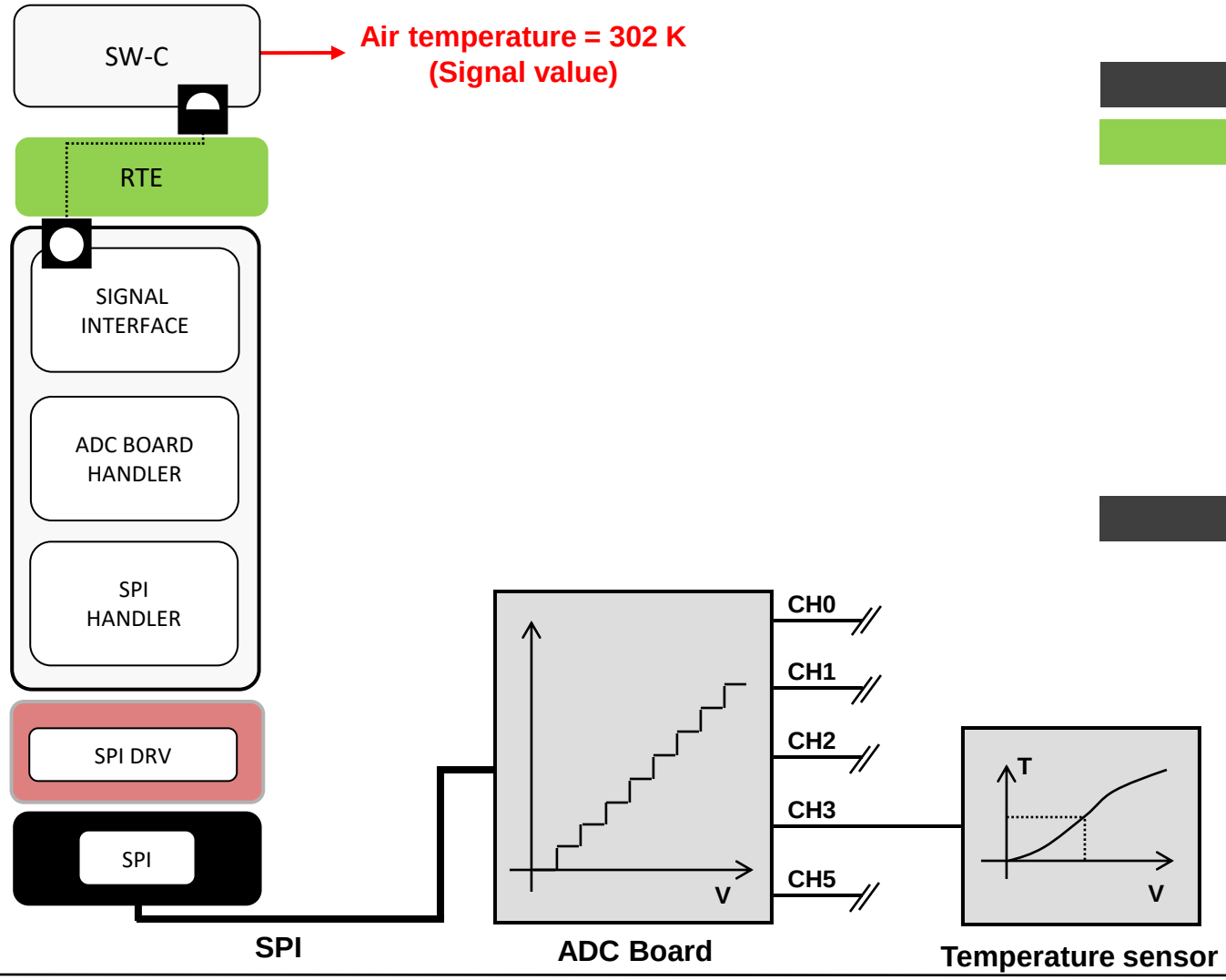
The AUTOSAR Architecture
Example 1: Hardware Abstraction

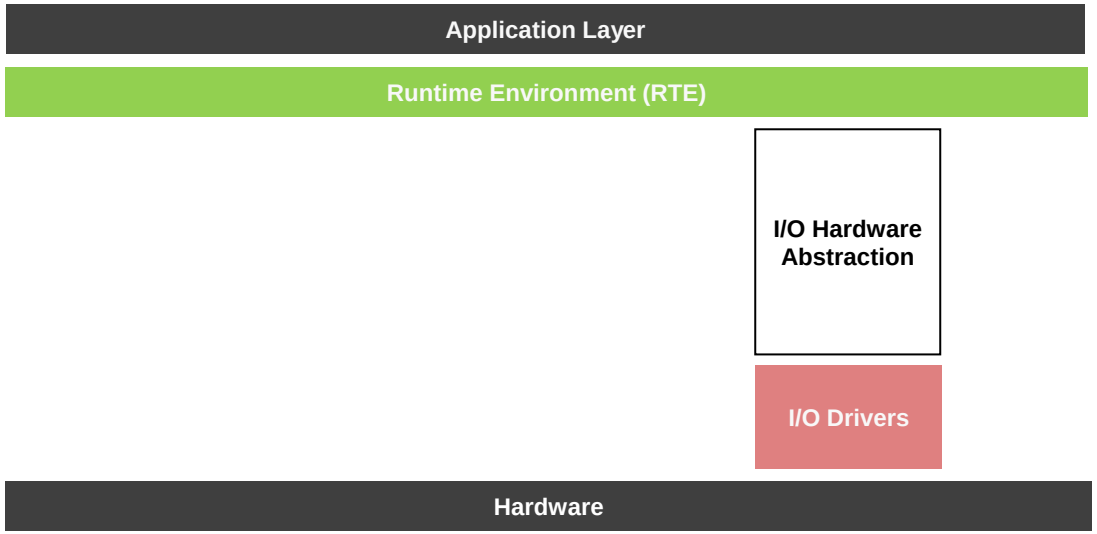
- Project specific
  - AUTOSAR provides high level requirements and guidelines



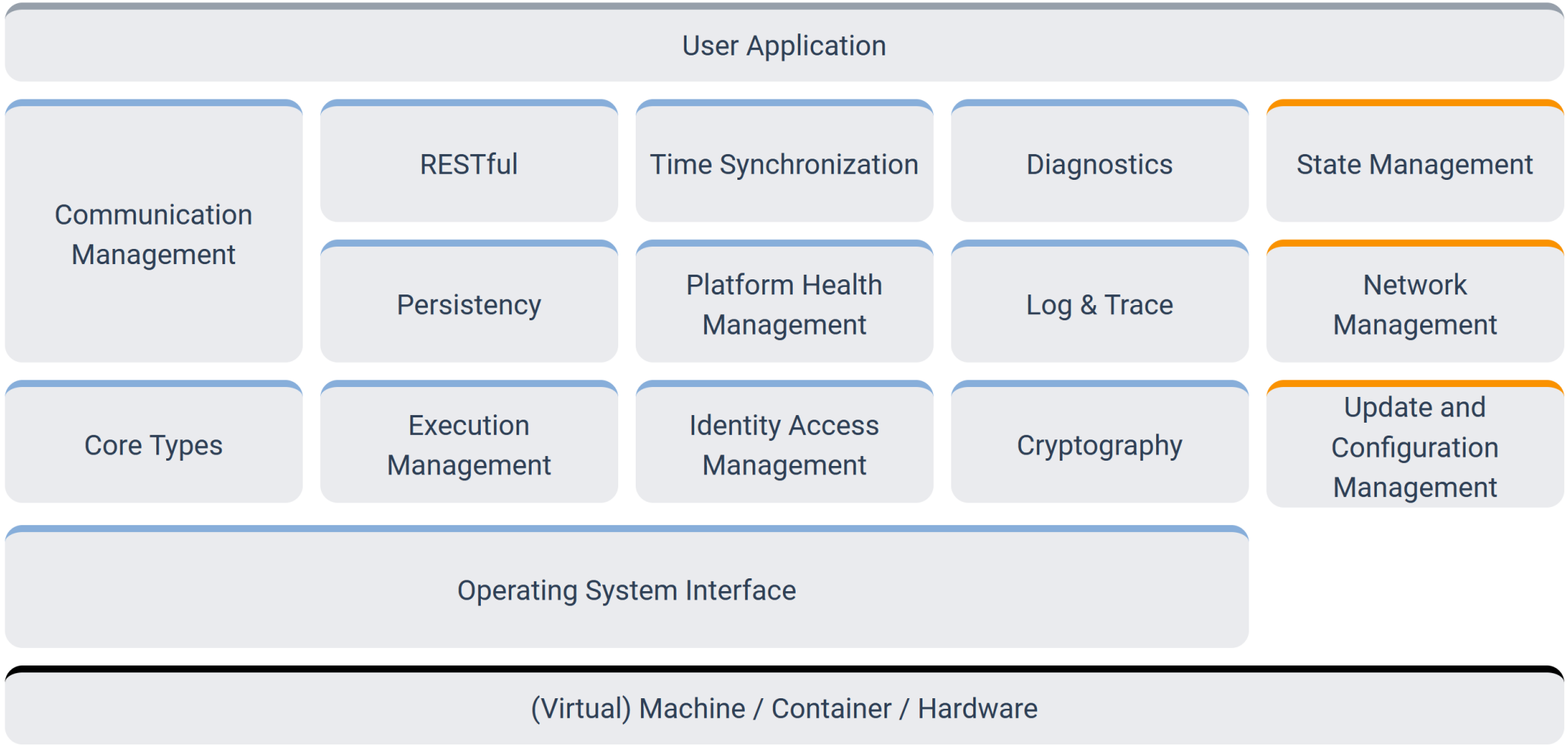
- Signal based interface
  - Digital IO
  - Analogue IO
  - PWM
- Encapsulates
  - Mapping signal to pin
  - Filtering and de-bouncing
  - Failure monitoring (SC to ground/power, open load, …) and reporting
  - Compensation of static influences
  - Conversation to physical units
  - Handling of SPI driven devices

The AUTOSAR Architecture
Example 1: Hardware Abstraction





The AUTOSAR Adaptive Platform (Release: R20-11)

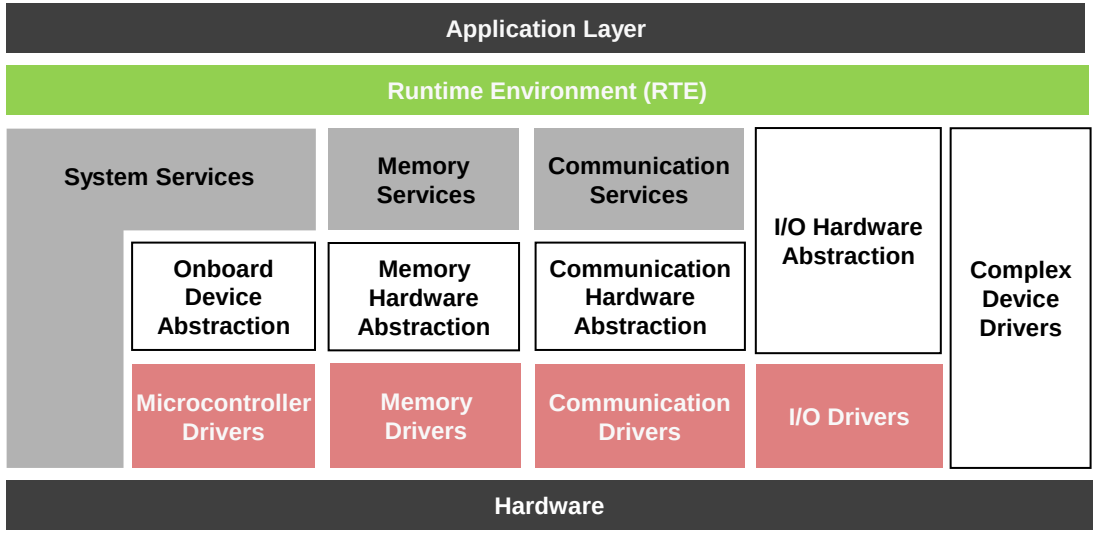


Source: https://www.autosar.org/standards/adaptive-platform/

Summary

Summary

- AUTOSAR provides standardized interfaces to applications
- Main Workgroups: Architecture, Methodology, Application Interfaces, Acceptance Tests
- AUTOSAR Platforms: Classic (Static) and Adaptive (Dynamic)
- SecOC module provides authentication mechanisms to detect attacks such as replay, spoofing and tampering



Literature

Literature

[1] https://www.autosar.org/, accessed 24.11.2021

[2] EB TRESOS AUTOSAR https://www.elektrobit.com/products/ecu/technologies/autosar/, accessed 24.02.2019

[3] M. Dworkin: Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication, U.S. Department of Commerce, Information Technology Laboratory (ITL), National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, NIST Special Publication 800-38B, 2005

[4] Specification of Secure Onboard Communication, AUTOSAR CP Release 4.3.1, https://www.autosar.org/fileadmin/user_upload/standards/classic/4-3/AUTOSAR_SWS_SecureOnboardCommunication.pdf, accessed 24.02.2019